Assignment 1: Aggregation

1. Total revenue generated (sum of totalAmount):

```
[
  {
    $group: {
      _id: null,
      totalRevenue: { $sum: "$totalAmount" }
    }
  }
]
```

---

2. Total number of orders by status:

```
[
  {
    $group: {
      _id: "$status",
      orderCount: { $sum: 1 }
    }
  }
]
```

---

3. Top 3 customers who spent the most:

```
[
  {
    $group: {
      _id: "$customerId",
      totalSpent: { $sum: "$totalAmount" }
    }
  },
  { $sort: { totalSpent: -1 } },
  { $limit: 3 }
]
```

---

4. Average order amount per customer:

```
[
  {
    $group: {
      _id: "$customerId",
      averageOrderAmount: { $avg: "$totalAmount" }
    }
  }
]
```

---

5. Products sold more than 10 times (by total quantity):

```
[
  { $unwind: "$items" },
  {
    $group: {
      _id: "$items.productId",
      totalQuantitySold: { $sum: "$items.quantity" }
    }
  },
  { $match: { totalQuantitySold: { $gt: 10 } } }
]
```

---

6. Monthly revenue for the last 6 months:

```
[
  {
    $addFields: {
      monthYear: {
        $dateToString: { format: "%Y-%m", date: "$orderDate" }
      }
    }
  },
  {
    $group: {
      _id: "$monthYear",
      monthlyRevenue: { $sum: "$totalAmount" }
    }
  },
  { $sort: { _id: -1 } },
  { $limit: 6 },
  { $sort: { _id: 1 } }
]
```

---
7. Customers who placed more than 2 orders:

```
[
  {
    $group: {
      _id: "$customerId",
      orderCount: { $sum: 1 }
    }
  },
  { $match: { orderCount: { $gt: 2 } } }
]
```

---
8. Extract only product names using $unwind and $project:

```
[
  { $unwind: "$items" },
  {
    $project: {
      _id: 0,
      productName: "$items.productName"
    }
  }
]
```
---
9. Filter only Delivered orders and calculate revenue:

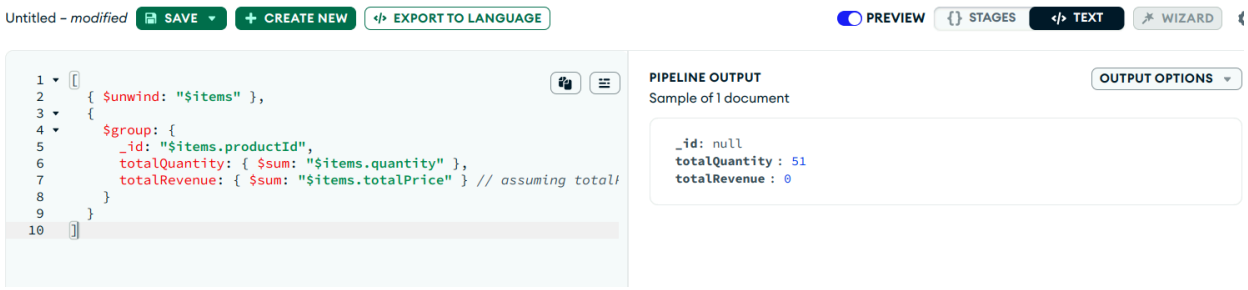
```
[
  { $match: { status: "Delivered" } },
  {
    $group: {
      _id: null,
      deliveredRevenue: { $sum: "$totalAmount" }
    }
  }
]
```

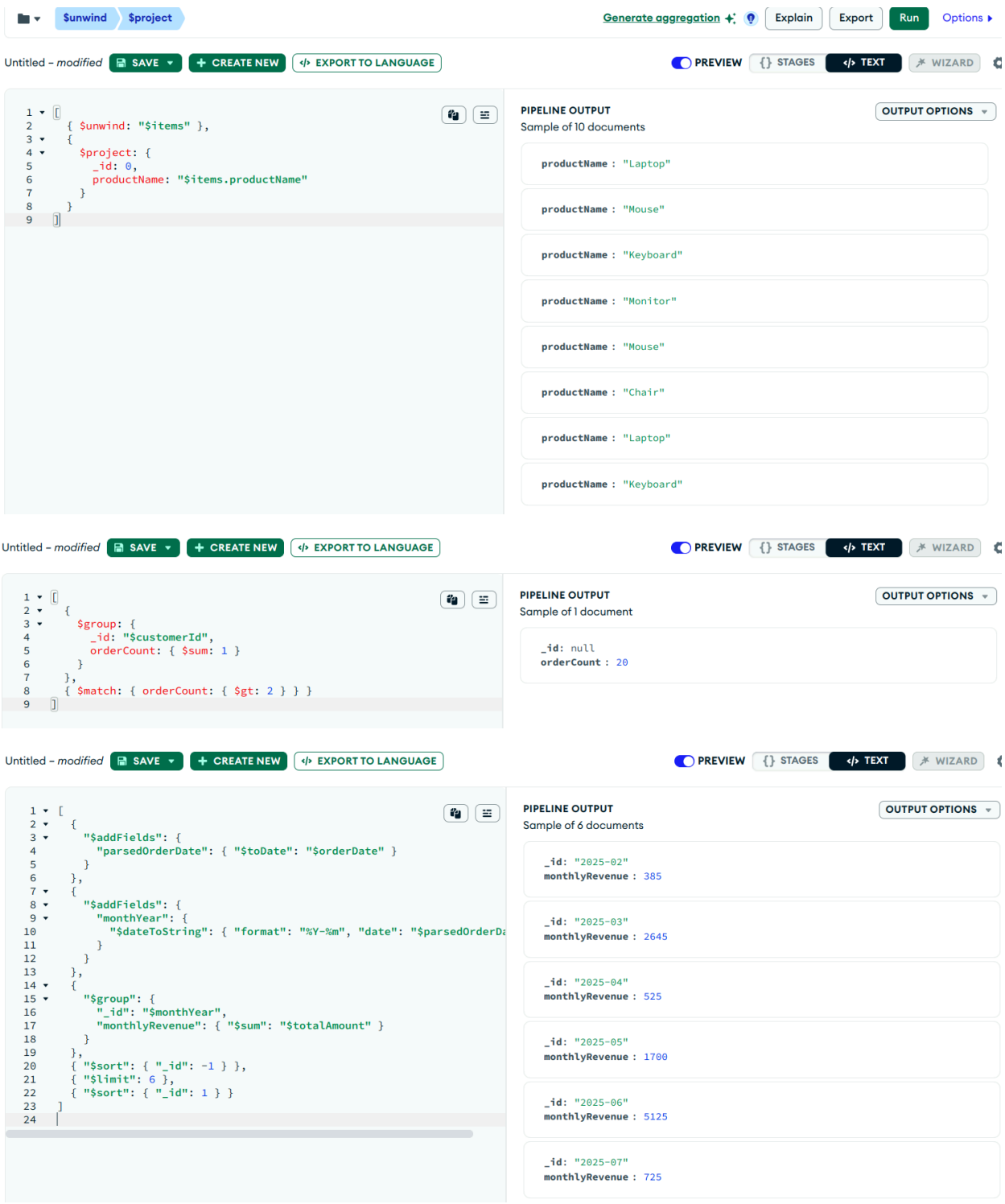
---
10. Total quantity and total revenue per product:

```
[
  { $unwind: "$items" },
```

```
  {
    $group: {
      _id: "$items.productId",
      totalQuantity: { $sum: "$items.quantity" },
      totalRevenue: { $sum: "$items.totalPrice" } // assuming totalPrice is in items
    }
  }
]
```

Screenshots

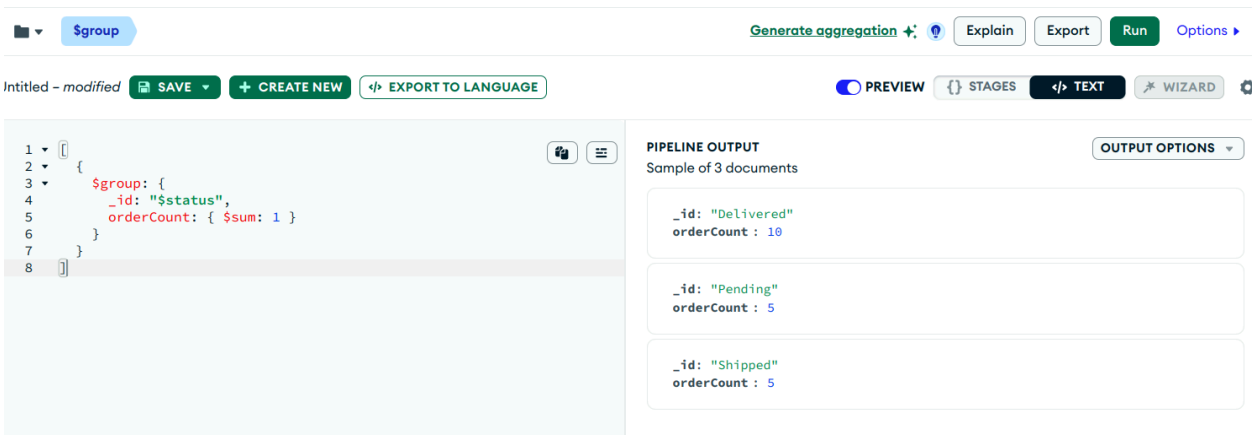


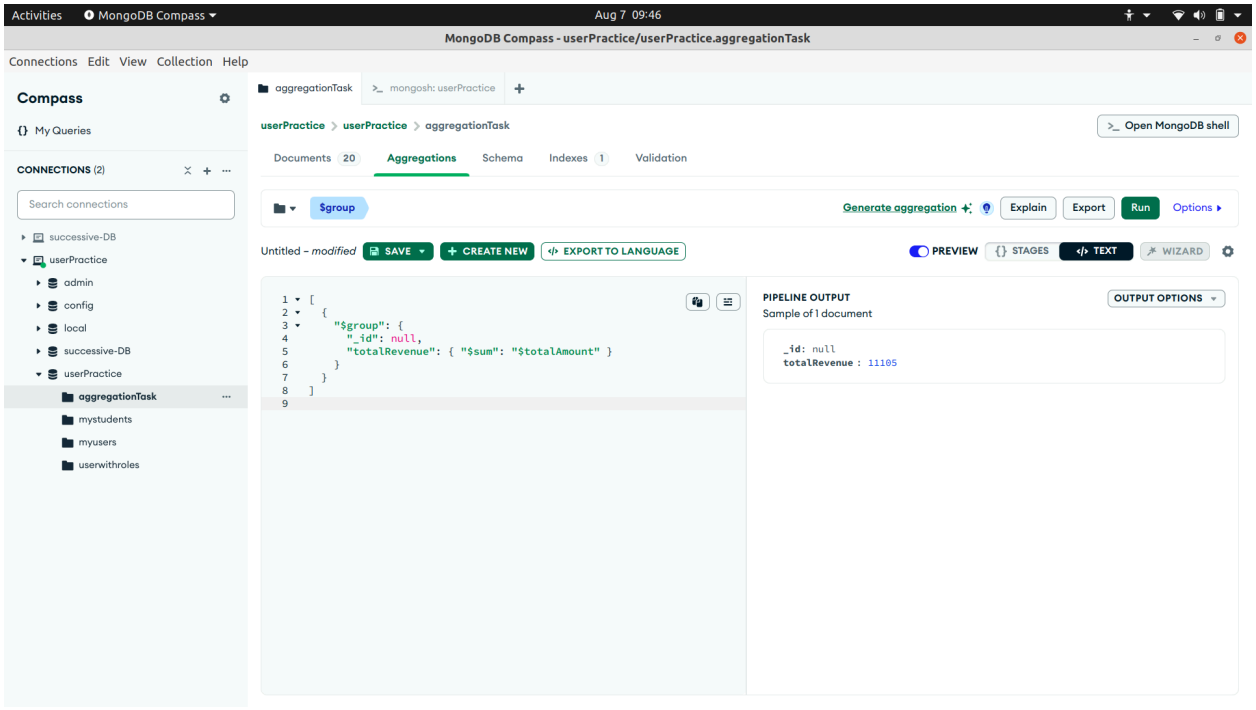

$unwind
$project
Generate aggregation
Explain
Export
Run
Options
Untitled – modified
SAVE
CREATE NEW
EXPORT TO LANGUAGE
PREVIEW
STAGES
TEXT
WIZARD
[
{ $unwind: "$items" },
{
$project: {
_id: 0,
productName: "$items.productName"
}
}
]
PIPELINE OUTPUT
Sample of 10 documents
OUTPUT OPTIONS
productName : "Laptop"
productName : "Mouse"
productName : "Keyboard"
productName : "Monitor"
productName : "Mouse"
productName : "Chair"
productName : "Laptop"
productName : "Keyboard"
Untitled – modified
SAVE
CREATE NEW
EXPORT TO LANGUAGE
PREVIEW
STAGES
TEXT
WIZARD
[
{
$group: {
_id: "$customerId",
orderCount: { $sum: 1 }
}
},
{ $match: { orderCount: { $gt: 2 } } }
]
PIPELINE OUTPUT
Sample of 1 document
OUTPUT OPTIONS
_id: null
orderCount : 20
Untitled – modified
SAVE
CREATE NEW
EXPORT TO LANGUAGE
PREVIEW
STAGES
TEXT
WIZARD
[
{
"$addFields": {
"parsedOrderDate": { "$toDate": "$orderDate" }
}
},
{
"$addFields": {
"monthYear": {
"$dateToString": { "format": "%Y-%m", "date": "$parsedOrderDa
}
}
},
{
"$group": {
"_id": "$monthYear",
"monthlyRevenue": { "$sum": "$totalAmount" }
}
},
{ "$sort": { "_id": -1 } },
{ "$limit": 6 },
{ "$sort": { "_id": 1 } }
]
PIPELINE OUTPUT
Sample of 6 documents
OUTPUT OPTIONS
_id: "2025-02"
monthlyRevenue : 385
_id: "2025-03"
monthlyRevenue : 2645
_id: "2025-04"
monthlyRevenue : 525
_id: "2025-05"
monthlyRevenue : 1700
_id: "2025-06"
monthlyRevenue : 5125
_id: "2025-07"
monthlyRevenue : 725


Untitled – modified
SAVE
CREATE NEW
EXPORT TO LANGUAGE
PREVIEW
STAGES
TEXT
WIZARD
[
{ $unwind: "$items" },
{
$group: {
_id: "$items.productId",
totalQuantitySold: { $sum: "$items.quantity" }
}
},
{ $match: { totalQuantitySold: { $gt: 10 } } }
]
PIPELINE OUTPUT
Sample of 1 document
OUTPUT OPTIONS
_id: null
totalQuantitySold : 51


$group
Generate aggregation
Explain
Export
Run
Options
Untitled – modified
SAVE
CREATE NEW
EXPORT TO LANGUAGE
PREVIEW
STAGES
TEXT
WIZARD
[
{
$group: {
_id: "$customerId",
averageOrderAmount: { $avg: "$totalAmount" }
}
}
]
PIPELINE OUTPUT
Sample of 1 document
OUTPUT OPTIONS
_id: null
averageOrderAmount : 555.25


$group
$sort
$limit
Generate aggregation
Explain
Export
Run
Options
Untitled – modified
SAVE
CREATE NEW
EXPORT TO LANGUAGE
PREVIEW
STAGES
TEXT
WIZARD
[
{
$group: {
_id: "$customerId",
totalSpent: { $sum: "$totalAmount" }
}
},
{ $sort: { totalSpent: -1 } },
{ $limit: 3 }
]
PIPELINE OUTPUT
Sample of 1 document
OUTPUT OPTIONS
_id: null
totalSpent : 11105


$group
Generate aggregation
Explain
Export
Run
Options
ntitled – modified
SAVE
CREATE NEW
EXPORT TO LANGUAGE
PREVIEW
STAGES
TEXT
WIZARD
[
{
$group: {
_id: "$status",
orderCount: { $sum: 1 }
}
}
]
PIPELINE OUTPUT
Sample of 3 documents
OUTPUT OPTIONS
_id: "Delivered"
orderCount : 10
_id: "Pending"
orderCount : 5
_id: "Shipped"
orderCount : 5



Assignment 2 : Indexing

1. Check indexes on the collection
db.aggregationTask.getIndexes()

2.Run query without index
db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")

3.Create index
db.aggregationTask.createIndex({ customerName: 1 })

4.now run again with index
db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")

---

5. Create compound index
Run query without compound index
db.aggregationTask.find({ status: "Delivered", orderDate: { $gte: ISODate("2025-04-01") }
}).explain("executionStats");

6.after Creating compound index
db.aggregationTask.createIndex({ status: 1, orderDate: -1 })
db.aggregationTask.find({ status: "Delivered", orderDate: { $gte: ISODate("2025-04-01") }
}).explain("executionStats")

7.Text index on items.productName
Create and use text index
db.aggregationTask.createIndex({ "items.productName": "text" })
db.aggregationTask.find({ $text: { $search: "Laptop" } })

8.Drop an index and compare performance
db.aggregationTask.dropIndex({ customerName: 1 })
db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")

Screenshots :

```
userPractice> db.aggregationTask.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
userPractice>
```

```
userPractice> db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'userPractice.aggregationTask',
    parsedQuery: { customerName: { '$eq': 'John Doe' } },
    indexFilterSet: false,
    queryHash: '17B49A18',
    planCacheShapeHash: '17B49A18',
    planCacheKey: '56634808',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { customerName: { '$eq': 'John Doe' } },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 20,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { customerName: { '$eq': 'John Doe' } },

userPractice> db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'userPractice.aggregationTask',
    parsedQuery: { customerName: { '$eq': 'John Doe' } },
    indexFilterSet: false,
    queryHash: '17B49A18',
    planCacheShapeHash: '17B49A18',
    planCacheKey: '56634808'
```

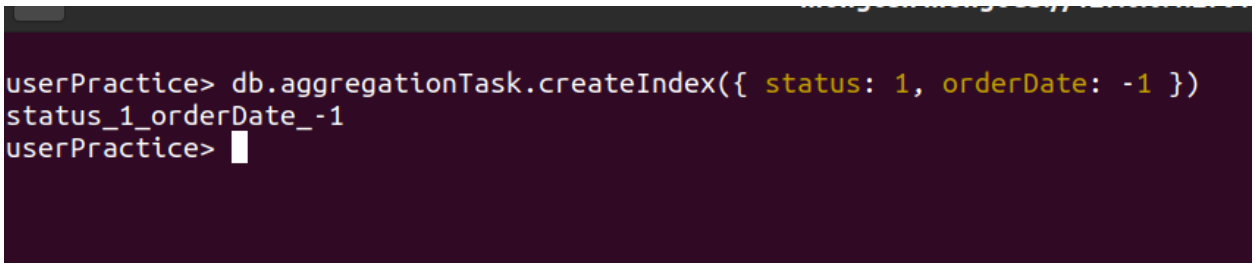
```
userPractice> db.aggregationTask.createIndex({ customerName: 1 })
customerName_1
userPractice>
```

```
serPractice> db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")
{
 explainVersion: '1',
 queryPlanner: {
   namespace: 'userPractice.aggregationTask',
   parsedQuery: { customerName: { '$eq': 'John Doe' } },
   indexFilterSet: false,
   queryHash: '17B49A18',
   planCacheShapeHash: '17B49A18',
   planCacheKey: '56634808',
   optimizationTimeMillis: 0,
   maxIndexedOrSolutionsReached: false,
   maxIndexedAndSolutionsReached: false,
   maxScansToExplodeReached: false,
   prunedSimilarIndexes: false,
   winningPlan: {
     isCached: false,
     stage: 'COLLSCAN',
     filter: { customerName: { '$eq': 'John Doe' } },
     direction: 'forward'
   },
   rejectedPlans: []
 },
 executionStats: {
   executionSuccess: true,
   nReturned: 1,
   executionTimeMillis: 0,
   totalKeysExamined: 0,
   totalDocsExamined: 20,
   executionStages: {
     isCached: false,
     stage: 'COLLSCAN',
     filter: { customerName: { '$eq': 'John Doe' } },

serPractice> db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")
{
 explainVersion: '1',
 queryPlanner: {
   namespace: 'userPractice.aggregationTask',
   parsedQuery: { customerName: { '$eq': 'John Doe' } },
   indexFilterSet: false,
   queryHash: '17B49A18',
   planCacheShapeHash: '17B49A18',
   planCacheKey: '56634808',
   optimizationTimeMillis: 0,
```

```
userPractice> db.aggregationTask.find({ status: "Delivered", orderDate: { $gte: ISODate("2025-04-01") } }).explain("executionStats");
{
 explainVersion: '1',
 queryPlanner: {
   namespace: 'userPractice.aggregationTask',
   parsedQuery: {
     '$and': [
       { status: { '$eq': 'Delivered' } },
       { orderDate: { '$gte': ISODate('2025-04-01T00:00:00.000Z') } }
     ]
   },
   indexFilterSet: false,
   queryHash: '888CD4EF',
   planCacheShapeHash: '888CD4EF',
   planCacheKey: 'CDDEBD83',
   optimizationTimeMillis: 0,
   maxIndexedOrSolutionsReached: false,
   maxIndexedAndSolutionsReached: false,
   maxScansToExplodeReached: false,
   prunedSimilarIndexes: false,
   winningPlan: {
     isCached: false,
     stage: 'COLLSCAN',
     filter: {
       '$and': [
         { status: { '$eq': 'Delivered' } },
         {
           orderDate: { '$gte': ISODate('2025-04-01T00:00:00.000Z') }
         }
       ]
     },
     direction: 'forward'
   },
   rejectedPlans: []
 },
 executionStats: {
   executionSuccess: true,
   nReturned: 0,
   executionTimeMillis: 0,
   totalKeysExamined: 0,
   totalDocsExamined: 20,
   executionStages: {
     isCached: false,
     stage: 'COLLSCAN',
     filter: {
```


mongosh mongodb://127.0.0.1:27017/?directConnection=
executionSuccess: true,
nReturned: 0,
executionTimeMillis: 0,
totalKeysExamined: 0,
totalDocsExamined: 20,
executionStages: {
isCached: false,
stage: 'COLLSCAN',
filter: {
'$and': [
{ status: { '$eq': 'Delivered' } },
{
orderDate: { '$gte': ISODate('2025-04-01T00:00:00.000Z') }
}
]
},
nReturned: 0,
executionTimeMillisEstimate: 0,
works: 21,
advanced: 0,
needTime: 20,
needYield: 0,
saveState: 0,
restoreState: 0,
isEOF: 1,
direction: 'forward',
docsExamined: 20
}
},
queryShapeHash: 'F36B2C35E0206E99C1E2A1EEF689AC281869ABC9C70EF9A08602BC8903C24C78',
command: {
find: 'aggregationTask',
filter: {
status: 'Delivered',
orderDate: { '$gte': ISODate('2025-04-01T00:00:00.000Z') }
},
'$db': 'userPractice'
},
serverInfo: {
host: 'viratssvrs-12.successivetech.com',
port: 27017,
version: '8.0.10',
gitVersion: '9d03076bb2d5147d5b6fe381c7118b0b0478b682'
},
serverParameters: {


userPractice> db.aggregationTask.createIndex({ status: 1, orderDate: -1 })
status_1_orderDate_-1
userPractice>

```
userPractice> db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'userPractice.aggregationTask',
    parsedQuery: { customerName: { '$eq': 'John Doe' } },
    indexFilterSet: false,
    queryHash: '17B49A18',
    planCacheShapeHash: '17B49A18',
    planCacheKey: '56634808',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { customerName: { '$eq': 'John Doe' } },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 20,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { customerName: { '$eq': 'John Doe' } },

userPractice> db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'userPractice.aggregationTask',
    parsedQuery: { customerName: { '$eq': 'John Doe' } },
    indexFilterSet: false,
    queryHash: '17B49A18',
    planCacheShapeHash: '17B49A18',
    planCacheKey: '56634808',
    optimizationTimeMillis: 0
```

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: { customerName: [ '["John Doe", "John Doe"]' ] }
    }
  },
  rejectedPlans: []
 },
 executionStats: {
  executionSuccess: true,
  nReturned: 1,
  executionTimeMillis: 1,
  totalKeysExamined: 1,
  totalDocsExamined: 1,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 1,
    executionTimeMillisEstimate: 0,
    works: 2,
    advanced: 1,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 1,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 2,
      advanced: 1,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,

userPractice> db.aggregationTask.find({ status: "Delivered", orderDate: { $gte: ISODate("2025-04-01") } }).explain("executionStats");
{
  explainVersion: '1',
  queryPlanner: {
```

```
userPractice> db.aggregationTask.find({ status: "Delivered", orderDate: { $gte: ISODate("2025-04-01") } }).explain("executionStats");
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'userPractice.aggregationTask',
    parsedQuery: {
      '$and': [
        { status: { '$eq': 'Delivered' } },
        { orderDate: { '$gte': ISODate('2025-04-01T00:00:00.000Z') } }
      ]
    },
    indexFilterSet: false,
    queryHash: '888CD4EF',
    planCacheShapeHash: '888CD4EF',
    planCacheKey: 'CDDEBD83',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        '$and': [
          { status: { '$eq': 'Delivered' } },
          {
            orderDate: { '$gte': ISODate('2025-04-01T00:00:00.000Z') }
          }
        ]
      },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 0,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 20,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
```

```
userPractice> db.aggregationTask.createIndex({ "items.productName": "text" })
items.productName_text
userPractice>
```

```
userPractice> db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'userPractice.aggregationTask',
    parsedQuery: { customerName: { '$eq': 'John Doe' } },
    indexFilterSet: false,
    queryHash: '17B49A18',
    planCacheShapeHash: '17B49A18',
    planCacheKey: '56634808',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { customerName: { '$eq': 'John Doe' } },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 20,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { customerName: { '$eq': 'John Doe' } },

userPractice> db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'userPractice.aggregationTask',
    parsedQuery: { customerName: { '$eq': 'John Doe' } },
    indexFilterSet: false,
    queryHash: '17B49A18',
    planCacheShapeHash: '17B49A18',
    planCacheKey: '56634808',
```

```
userPractice> db.aggregationTask.dropIndex({ customerName: 1 })
{ nIndexesWas: 4, ok: 1 }
userPractice>
```

```
userPractice> db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'userPractice.aggregationTask',
    parsedQuery: { customerName: { '$eq': 'John Doe' } },
    indexFilterSet: false,
    queryHash: '17B49A18',
    planCacheShapeHash: '17B49A18',
    planCacheKey: '56634808',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { customerName: { '$eq': 'John Doe' } },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 20,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { customerName: { '$eq': 'John Doe' } },

userPractice> db.aggregationTask.find({ customerName: "John Doe" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'userPractice.aggregationTask',
    parsedQuery: { customerName: { '$eq': 'John Doe' } },
    indexFilterSet: false,
    queryHash: '17B49A18',
    planCacheShapeHash: '17B49A18',
    planCacheKey: '56634808',
    optimizationTimeMillis: 0,
```

Aggregation in MongoDB

Definition:

Aggregation is the process of processing data records and returning computed results. It allows you to perform operations like filtering, grouping, sorting, reshaping, and transforming data within the database — similar to SQL's `GROUP BY` and more.

MongoDB's aggregation framework uses a pipeline approach where documents pass through multiple stages, each stage transforming the data and passing it to the next.

Common Aggregation Stages

Stage	Purpose
`$match`	Filters documents (like `find()` query).
`$group`	Groups documents by a key and applies accumulators (`$sum`, `$avg`, etc.).
`$project`	Reshapes documents, include/exclude fields, add computed fields.
`$sort`	Sorts documents by specified field(s).
`$limit`	Limits number of documents in the output.
`$skip`	Skips over specified number of documents.

`$unwind`	**Deconstructs arrays into individual documents.**
`$addFields`	**Adds new fields or modifies existing fields.**
`$lookup`	**Performs a join with another collection.**

Use Cases

- **Generating reports (total sales, average order values)**
- **Data analysis (grouping customers by region)**
- **Data transformation (flattening nested arrays)**
- **Real-time dashboards and metrics**

Indexing in MongoDB

Definition:

An index is a special data structure that stores a small portion of the data set in an easy-to-traverse form. Indexes improve the speed of query operations by allowing MongoDB to quickly locate data without scanning the entire collection.

Common Commands

Command	Purpose
`db.collection.getIndexes()`	Lists all indexes on the collection.
`db.collection.createIndex()`	Creates a new index.
`db.collection.dropIndex()`	Drops an existing index by name.
`db.collection.explain()`	Explains query plan and index usage.

Use Cases

- Speeding up queries filtering on user ID, status, or date.
- Supporting text search for product names or descriptions.
- Enforcing uniqueness on email or username fields.
- Efficiently sorting large result sets.
- Automatically deleting expired sessions with TTL indexes.